ADT G.729AB 8 KBPS Voice Compression Software User's Guide

Adaptive Digital Technologies, Inc.
November 29, 2012

Copyright © 2011 - 2015, Adaptive Digital Technologies Inc.

Revision History

Issue	Revision History	Date	Initials
1	Adapted from previous versions of this document	11/1/2011	SDK
2	XDM update	9/27/2012	CMW
3	XDM update	11/29/2012	CMW

Table of Contents

1. Introduction 1
1.1 References 2
1. G.729AB Data Sheet, Adaptive Digital **2**
2. ITU Recommendation G.729A **2**
3. ITU Recommendation G.729AB **2**
4. IETF RFC 3551 RTP Profile for Audio and Video Conferences With Minimal Control **2**
5. IETF RFC 3889 Real-time Transport Protocol (RTP) Payload for Comfort Noise (CN) **2**
2. Application Programming Interface 2
2.1 ADT "C" API 3
2.1.1 Instantiation 3
2.1.2 Initialization 3
2.1.2.1 Encoder Initialization 3
2.1.2.2 Decoder Initialization 4
2.1.2.3 Encoder 4
2.1.2.4 Decoder 5
2.1.3 Voice Activity Detection and Noise Insertion Descriptor Frames 6
2.1.3.1 Noise (VADFlag = 0) 6
2.1.3.2 Voice (VADFlag = 1) 7
2.1.3.3 Noise Insertion Descriptor (VADFlag = 2) 7
2.1.4 Sample Code 7
3. XDM API 8
3.1 Encoder Data Structures 8
3.2 Decoder Data Structures 10
3.3 Interface Functions 12
3.3.1 Creation 12
3.3.2 Processing 13
3.3.3 Control 14
3.3.4 Termination 15
3.4 Sample Code 16
3.5 C++ API 17
3.5.1 Sample Code 18

1. Introduction

The Adaptive Digital Technologies (ADT) G.729A/B voice coder software is an implementation of ITU Recommendation G.729A/B 8 kbit/s CS-ACELP Speech Codec. The G.729 A/B voice coder provides near toll speech quality at a compressed data rate of 8 kbit/s: a 16:1 compression ratio.

This document describes Adaptive Digital's G.729AB algorithm implementation. This document includes the characteristics that are common across Adaptive Digital's G.729AB offerings on many different processors. For information that is processor-specific such as MIPS and Memory specifications, deliverable information, and ordering information, please refer to Adaptive Digital's G.729AB data sheet, which can be found at www.adaptivedigital.com. Alternately, you may request a copy from Adaptive Digital via e-mail: information@adaptivedigital.com.

G.729 Annex A (G.729A) is a reduced complexity version of G.729. It has the same compression characteristics, but sacrifices a small amount of voice quality in return for nearly half the complexity.

G.729 Annex B (G.729B, or G.729AB when both annex A and B combined) specifies is a method to further reduce the bit rate by taking advantage of non-speech periods in the input signal. The technique is known as Discontinuous Transmission (DTX). The G.729AB encoder can output 3 different types of compressed outputs depending upon the voice activity:

> **Voice**: In this case, voice was detected and a full 80 bits per frame (8 kbps) are produced
> **SID (Silence Identifier)**: In this case, silence or background noise was detected. 16 bits per frame (1.6 kbps) are produced. The premise is that fewer bits are necessary to model noise as compared with modeling speech.
> **Null**: In this case, noise was detected, but the characteristics have not changed significantly compared to the most recent SID frame. No data is produced.

The first non-voice frame that follows a voice frame is always a SID frame. The reason is that it is necessary to let the decoder know the background noise characteristics after the end of speech spurt because the decoder may be asked to continue to generate noise in the absence of new input data (Null Frames).

The decoder handles the 3 frame types as follows:

> **Voice**: The decoder performs a normal G.729A decode
> **SID (Silence Identifier)**: The decoder updates its noise model based upon the 16 bits of compressed data, and generates noise based upon this model.
> **Null**: The null frame is a bit more complicated. Since no data is received at all, the Decoder must decide if the frame is just a non-voiced period or if the data was lost before it reached the decoder (or simply did not arrive in time to decode it.) This can happen in packet networks. The application code that controls the decoder should handle this situation as follows:

```
If no data was received in the current frame
    If the most recently received frame was a voice frame
        Declare a frame loss. The decoder will try to fill in the lost frame.
    Else
        Treat the current frame as a null frame. Generate noise using the
        noise characteristics contained in the most recently received SID
        frame.
```

Since the encoder is never allowed to follow a voice frame with a null frame, the decoder must declare a frame loss whenever a null follows a voice frame.

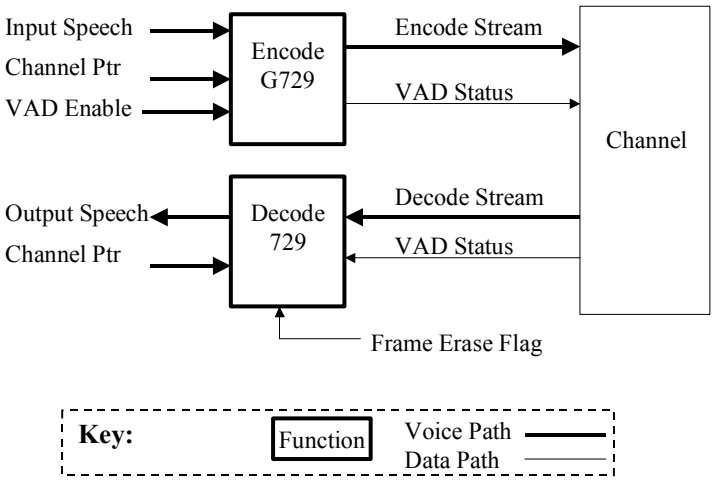
In systems that employ bit error detection, the application may decide to declare a lost frame during frames with excessive bit errors in order to prevent undesired decoder outputs due to the bit errors.

1.1 References

1. G.729AB Data Sheet, Adaptive Digital
2. ITU Recommendation G.729A
3. ITU Recommendation G.729AB
4. IETF RFC 3551 RTP Profile for Audio and Video Conferences With Minimal Control
5. IETF RFC 3889 Real-time Transport Protocol (RTP) Payload for Comfort Noise (CN)

2. Application Programming Interface

The G.729A/B voice channel is show in the block diagram below.



Adaptive Digital's G.729AB can be used via a number of API types:

- ADT “C” API
- XDM API
- ADT “C++” API

Each of these APIs is described in the sections that follow. It should be noted that not all implementations support all the APIs. For example, the C++ API is currently limited to PC/Windows implementations.

2.1 ADT “C” API

2.1.1 Instantiation

The host software is required to allocate persistent memory for each Encoder and Decoder channel (instance).

In addition, the host software must allocate scratch memory for the encoder and decoder. Depending on the host environment, the encode and decode scratch may be overlaid. If preemption is not used at all, the encoder and decode scratch can be overlaid and only one copy of scratch is needed for all channels.

The scratch memory requirements are listed in section 2 of this document.

The following example shows the instantiation of multiple (N) channels persistent (per-channel) and scratch memory for the encoder and decoder. In this example, the encoder and decoder have separate declarations to allow the encoder and decoder to preempt each other.

```
G729ENC_ADT_Instance_t EncState[N];
G729ENC_ADT_Scratch_t EncScratch;
G729DEC_ADT_Instance_t DecState[N];
G729DEC_ADT_Scratch_t DecScratch;
```

If the encoder and decoder cannot preempt each other, one could change the instantiation as follows:

```
G729ENC_ADT_Instance_t EncState[N];
G729DEC_ADT_Instance_t DecState[N];
union
{
        G729ENC_ADT_Scratch_t EncScratch;
        G729DEC_ADT_Scratch_t DecScratch;
}               EncDecScratch;
```

2.1.2 Initialization

An initialization function is provided to initialize the G.729 vocoder encoder and decoder. The functions are described in the following sections.

2.1.2.1 Encoder Initialization

Prototype:

```
void G729AB_ADT_EncodeInit(
        G729ENC_ADT_Instance_t *Channel,
        G729ENC_ADT_Scratch_t *scratch);
```

Inputs:

Channel – points to the G.729AB encoder instance's persistent memory structure
scratch – points to the scratch structure

2.1.2.2 Decoder Initialization

Prototype:

```
void G729AB_ADT_DecodeInit(
        G729DEC_ADT_Instance_t *Channel,
        G729DEC_ADT_Scratch_t *scratch);
```

Inputs:

Channel – points to the G.729AB decoder instance's persistent memory structure
scratch – points to the scratch structure

2.1.2.3 Encoder

The encoder can generate either of two forms of compressed data:

1. Packed bytes
2. G.729AB Parameter Array (per ITU G.729AB)

Depending upon which output format is desired, the user must call one of the two following functions:

G729AB_ADT_Encode generates packed bytes
G729AB_ADT_Encode_To_Params generates the G.729AB parameter array

Prototype:

```
void G729AB_ADT_Encode(
        G729ENC_ADT_Instance_t *Channel,
        short int *Speech,
        char *Compressed,
        short int VADEnable,
        short int *VADFlag
);
```

Input:

Channel – Pointer to the channel structure
Speech [ ] – The input speech data consisting of 80 uniformly quantized 16-bit samples
VADEnable – This bit, when set, enables the G.729B voice activity detection and silence suppression. This flag is only valid if the licensed software includes the G.729B portion

Output:

compressed [ ] – The G.729 compressed output data packed into up to 10 bytes. The compressed data conforms to the RTP payload format specified by RFC 3551 (for voice) and by RFC3889 (for SID packets)

VADFlag – "0" indicates that voice activity is not present and no noise updates are necessary. "1" indicates that voice activity was detected. "2" indicates that a new silence identifier frame is available. This flag word is only used if the licensed software includes the G.729B portion.

Prototype:

```
void G729AB_ADT_Encode_To_Params(
        G729ENC_ADT_Instance_t *Channel,
        Word16 *Speech,
        Word16 *Params,
        Word16 VADEnable,
        Word16 *VADFlag
        );
```

Input:

Channel – Pointer to the channel structure

Speech [ ] – The input speech data consisting of 80 uniformly quantized 16-bit samples

VADEnable – This bit, when set, enables the G.729B voice activity detection and silence suppression. This flag is only valid if the licensed software includes the G.729B portion

Output:

Params – The G.729 compressed output parameters. This array is of size 13 16-bit words.

VADFlag – "0" indicates that voice activity is not present and no noise updates are necessary. "1" indicates that voice activity was detected. "2" indicates that a new silence identifier frame is available.

2.1.2.4 Decoder

Prototype:

```
void G729AB_ADT_Decode(
        G729DEC_ADT_Instance_t *Channel,
        char *Compressed,
        short int *Speech,
        short int FrameErase,
        short int VADFlag
);
```

Input:

Channel – Pointer to the channel structure

compressed [ ] – The G.729 compressed input data packed into up to 10 bytes. The compressed data conforms to the RTP payload format specified by RFC 3551 (for voice) and by RFC3889 (for SID packets)

FrameErase – If set, this flag informs the decoder that the Stream array has no valid data. The decoder should handle the frame as an erasure frame. If not set, the decoder should operate normally.

VADFlag – "0" indicates that voice activity is not present and no noise updates are necessary. "1" indicates that voice activity was detected. "2" indicates that a new silence identifier frame is available. This flag word is only used if the licensed software includes the G.729B portion.

Output:

speech [ ] – The synthesized output speech data consisting of 80 uniformly quantized 16-bit samples.

Prototype:

```
void G729AB_ADT_Decode_From_Param(
        G729DEC_ADT_Instance_t *Channel,
        short int *Param,
        short int *Speech,
        short int FrameErase,
        short int VADFlag
);
```

Input:

Channel – Pointer to the channel structure

Param [ ] – The G.729AB encoded Parameters. This array is of length 13 16-bit words is in the parameter format set forth by ITU recommendations G.729 Annex A and G.729 Annex B.

FrameErase – If set, this flag informs the decoder that the Stream array has no valid data. The decoder should handle the frame as an erasure frame. If not set, the decoder should operate normally.

VADFlag – "0" indicates that voice activity is not present and no noise updates are necessary. "1" indicates that voice activity was detected. "2" indicates that a new silence identifier frame is available.

Output:

speech [ ] – The synthesized output speech data consisting of 80 uniformly quantized 16-bit samples.

2.1.3 Voice Activity Detection and Noise Insertion Descriptor Frames

Annex B of the ITU G.729 vocoder has the capability of detecting voice activity and inserting comfort noise in the absence of voice activity. When comfort noise is being generated, the characteristics of the generated noise at the decode side are similar to that of the actual background noise at the encode side. In order to achieve this, it is necessary to update the comfort noise generator parameters when the characteristics of the comfort noise change.

There are three VAD states reported by the encoder via the VADFlag word, Noise, Voice, and Noise Descriptor. The significance of these VAD states is described in the sections that follow.

2.1.3.1 Noise (VADFlag = 0)

No voice activity detected and no comfort noise parameter updates are necessary. The decoder will continue to generate comfort noise using the most recent comfort noise parameters. When the VADFlag is set to 0, no valid data is returned in the compressed[ ] array.

2.1.3.2 Voice (VADFlag = 1)

Voice activity was detected. The compressed[ ] array contains 80 bits, packed as either 10 bytes or 13 parameters (depending upon which encoder API is being used).

2.1.3.3 Noise Insertion Descriptor (VADFlag = 2)

No voice activity was detected. A change in the noise characteristics was detected. The compressed[ ] array contains either 2 bytes or 4 parameters (depending upon which decoder API is being used). This data, also known as the Noise Insertion Descriptor, contains updated noise parameters for use by the decoder comfort noise generator.

2.1.4 Sample Code

```
#include "g729ab_user.h"

// Instantiate the persistent (per-channel, or state) memory

G729ENC_ADT_Instance_t EncState;
G729DEC_ADT_Instance_t DecState;

// Instantiate the scratch memory

G729ENC_ADT_Scratch_t EncScratch;
G729DEC_ADT_Scratch_t DecScratch;

Word16 Speech[80];
char Packed[10];
Word16 VADEnable=1;
Word16 VADFlag;
Word16 FrameErase;
.
.
/* perform initialization at the start of each new voice call */

G729AB_ADT_EncodeInit(&EncState, &EncScratch);
G729AB_ADT_DecodeInit(&DecState, &DecScratch);
.
.
.
/* perform encode function. */

G729AB_ADT_Encode(&EncState, Speech, Packed, VADEnable, &VADFFlag);
.
.
.
/* perform decode function */

FrameErase = 0;

G729AB_ADT_Decode(&DecState, Packed, Speech, FrameErase, VADFlag);
```

3. XDM API

3.1 Encoder Data Structures

This section describes G.729AB Encoder specific extended data structures.

IG729ABENC_Params

Name	IG729ABENC_Params		
Description	This structure defines the creation parameters for all G729ABENC instance objects.		
Fields	**Datatype**	**Input/ output**	**Description**
isphenc1_params	ISPHENC1_Params	Input	All fields (except *size)* are not applicable to G.729AB Encoder

IG729ABENC_DynamicParams

Name	IG729ABENC_DynamicParams		
Description	This structure defines the parameters that can be modified after creation via control() calls.		
Fields	**Datatype**	**Input/ output**	**Description**
isphenc1_ DynamicParams	ISPHENC1_ DynamicParams	Input	All fields (except *size, vadFlag*) are not applicable to G.729AB Encoder *VadFlag:* when set, enables the G.729B voice activity detection and silence suppression.

IG729ABENC_InArgs

Name	IG729ABENC_InArgs		
Description	This structure defines run time input arguments used for the G.729AB encoder.		
Fields	**Datatype**	**Input/ output**	**Description**
isphenc1_inArgs	ISPHENC1_InArgs	Input	Run time input arguments
VADEnable	XDAS_Int16	Input	This bit, when set, enables the G.729B voice activity detection and silence suppression.
padding	XDAS_Int16	Input	for 32-bit alignment

Note:

Either *IG729ABENC_InArgs* or *ISPHENC1_InArgs* can be used.

If using *IG729ABENC_InArgs* the size of the input arguments should be equal to the size of *IG729ABENC_InArgs.* The parameter *VA*DEnable is passed to G.729AB encoder via parameter *IG729ABENC_InArgs.VADEnable*.

If using *ISPHENC1_InArgs* the size of the input arguments should be equal to the size of ISPHENC1_InArgs. Encode will be VAD enabled as default at initialization. It can be modified on-the-fly using the control API (*ISPHENC1_DynamicParams.vadFlag*).

IG729ABENC_ Status

Name	IG**729AB**ENC_Status		
Description	This structure defines the instance status parameters (read-only)		
Fields	**Datatype**	**Input/ output**	**Description**
isphenc1_status	ISPHENC1_Status	output	bufInfo: Input and output buffer information VadFlag: 1: VAD enabled 0: VAD disabled

IG729ABENC_OutArgs

Name	IG729ABENC_OutArgs		
Description	This structure defines run time output arguments used for the G.729AB encoder.		
Fields	**Datatype**	**Input/ output**	**Description**
isphenc1_out Args	ISPHENC1 _OutArgs	output	run time output arguments frameType – "2" indicates that voice activity is not present and no noise updates are necessary. "0" indicates that voice activity was detected. "1" indicates that a new silence identifier frame is available. extendedError -- Extended error information.

3.2 Decoder Data Structures

This section describes G.729AB Decoder specific extended data structures.

IG729ABDEC_Params

Name	IG729ABDEC_Params		
Description	This structure defines the creation parameters for all G729ABDEC instance objects.		
Fields	**Datatype**	**Input/ output**	**Description**
isphdec1_params	ISPHDEC1_Params	Input	All fields (except *size)* are not applicable to G.729AB Decoder

IG729ABDEC_DynamicParams

Name	IG729ABDEC_DynamicParams		
Description	This structure defines the parameters that can be modified after creation via control() calls.		
Fields	**Datatype**	**Input/ output**	**Description**
isphdec1_ DynamicParams	ISPHDEC1_ DynamicParams	Input	All fields (except *size)* are not applicable to G.729AB Decoder

IG729ABDEC_InArgs

Name	IG729ABDEC_InArgs		
Description	This structure defines run time input arguments used for the G.729AB decoder.		
Fields	**Datatype**	**Input/ output**	**Description**
isphdec1_inArgs	ISPHDEC1_InArgs	Input	run time input arguments frameType – "2" indicates that voice activity is not present and no noise updates are necessary. "0" indicates that voice activity was detected. "1" indicates that a new silence identifier frame is available.
frameErase	XDAS_Int16	Input	If set, the decoder handle the frame as an erasure frame. If not set, the decoder operates normally.
padding	XDAS_Int16	Input	For 32-bit alignment

Note:

Either *IG729ABDEC_InArgs* or *ISPHDEC1_InArgs* can be used.

If using *IG729ABDEC_InArgs* the size of the input arguments should be equal to the size of *IG729ABDEC _InArgs.* The parameters *frameErase* and *frameType* are passed to G.729AB decoder via parameters *IG729ABDEC_InArgs.FrameErase* and *IG729ABDEC_InArgs. isphdec1_inArgs.frameType.*

If using *ISPHDEC1_InArgs* the size of the input arguments should be equal to the size of *ISPHDEC1_InArgs.* The size of the input buffer should be set according to the frame types (Voice, SID, or Null, see section 1).

If (Voice)

inBufs_dec.bufSize = 10;

else if (SID)

inBufs_dec.bufSize = 2;

else

inBufs_dec.bufSize = 0;

IG729ABDEC_ Status

Name	IG**729AB**DEC_Status		
Description	This structure defines the instance status parameters (read-only)		
Fields	**Datatype**	**Input/ output**	**Description**
isphdec1_status	ISPHDEC1_Status	output	bufInfo: Input and output buffer information

IG729ABDEC_OutArgs

Name	IG729ABDEC_OutArgs		
Description	This structure defines run time output arguments used for the G.729AB decoder.		
Fields	**Datatype**	**Input/ output**	**Description**
isphdec1_outArgs	ISPHDEC1_OutArgs	output	run time output arguments extendedError -- Extended error information

3.3 Interface Functions

This section describes the Application Programming Interfaces (APIs) used in G.729AB encoder/decoder.

algNumAlloc(), algAlloc(), algInit(), algActivate(), algDeactivate(), and algFree() are standard XDAIS APIs. These functions are fully documented as part of the xDAIS specification.

3.3.1 Creation

algNumAlloc()

Synopsis:
```
XDAS_Int32 algNumAlloc(Void);
```

This function is implemented.

algAlloc()

Synopsis:
```
XDAS_Int32 algAlloc(const IALG_Params *params, IALG_Fxns
**parentFxns, IALG_MemRec memTab[]);
```

This function is implemented.

algInit()

Synopsis:
```
status = algInit(IALG_Handle handle, IALG_MemRec memTab[],
IALG_Handle parent, IALG_Params *params,);
```

This function initializes an instance of the G729AB Encode / Decode channel.

3.3.2 Processing

algActivate()

Synopsis:
```
Void algActivate(IALG_Handle handle);
```

This function is not implemented.

process()

Encoding call

Synopsis:
```
XDAS_Int32  (*process)(ISPHENC1_Handle handle,
        XDM1_SingleBufDesc *inBuf, XDM1_SingleBufDesc *outBuf,
        ISPHENC1_InArgs *inArgs, ISPHENC1_OutArgs *outArgs);
```

Arguments:
handle: Handle to an algorithm instance.
inBufs: Input buffer descriptors.
outBufs: Output buffer descriptors.
inArgs: Input arguments
outArgs: Ouput results

Return Value:

ISPHENC1_EOK	Success.
ISPHENC1_EFAIL	Failure.

Description:

ADT's implementation of the G.729AB encoder. The encoder can only generate one form of compressed data -- Packed bytes The compressed data conforms to the RTP payload format specified by RFC 3551 (for voice) and by RFC3889 (for SID packets).

process()

Decoding call

Synopsis:
```
XDAS_Int32  (*process)(ISPHDEC1_Handle handle,
        XDM1_SingleBufDesc *inBuf, XDM1_SingleBufDesc *outBuf,
        ISPHDEC1_InArgs *inArgs, ISPHDEC1_OutArgs *outArgs);
```

Arguments:
handle: Handle to an algorithm instance.
inBufs: Input buffer descriptors.
outBufs: Output buffer descriptors.
inArgs: Input arguments

outArgs: Ouput results

Return Value:
ISPHDEC1_EOK Success.
ISPHDEC1_EFAIL Failure.

Description:
ADT's implementation of the G.729AB decoder. The decoder can only receive one form of compressed data -- Packed bytes. The compressed data conforms to the RTP payload format specified by RFC 3551 (for voice) and by RFC3889 (for SID packets).

algDeactivate()

Synopsis:
```
Void algDeactivate(IALG_Handle handle);
```

This function is not implemented.

3.3.3 Control

Control()

Change run time parameters and query the status of the encoder

Synopsis:
```
XDAS_Int32  (*control) (ISPHENC1_Handle handle, ISPHENC1_Cmd id,
ISPHENC1_DynamicParams *params, ISPHENC1_Status *status);
```

Arguments:
Handle: Handle to an algorithm instance
id: Algorithm control command.
params: Dynamic parameters used to modify parameters as the algorithm runs.
status: Output status results.

Return Value:
ISPHENC1_EOK Success.
ISPHENC1_EFAIL Failure.

Description:
Control the object's parameters while the algorithm is running.

The command controls XDM_FLUSH, XDM_RESET and XDM_SETDEFAULT are not applicable to G.729AB ENCODER. The third and fourth arguments must be pointers to the extended DynamicParams and Status data structures respectively. Also, ensure that the size field is set to the size of the extended data structure.

Control()

Change run time parameters and query the status of the decoder

Synopsis:
```
XDAS_Int32  (*control) (ISPHDEC1_Handle handle, ISPHDEC1_Cmd id,
ISPHDEC1_DynamicParams *params, ISPHDEC1_Status *status);
```

Arguments:
Handle: Handle to an algorithm instance
id: Algorithm control command.
params: Dynamic parameters used to modify parameters as the algorithm runs.
status: Output status results.

Return Value:

ISPHDEC1_EOK	Success.
ISPHDEC1_EFAIL	Failure.

Description:
Control the object's parameters while the algorithm is running.

The command controls XDM_FLUSH, XDM_RESET and XDM_SETDEFAULT are not applicable to G.729AB DECODER. The third and fourth arguments must be pointers to the extended DynamicParams and Status data structures respectively. Also, ensure that the size field is set to the size of the extended data structure.

3.3.4 Termination

algFree()

Synopsis:
```
XDAS_Int32 algFree(IALG_Handle handle, IALG_MemRec memTab[]);
```

This function is implemented.

3.4 Sample Code

```
#include "g729abenc.h"
#include "g729abenc_adt.h"
#include "g729abdec.h"
#include "g729abdec_adt.h"

#define L_FRAME 80
Word16 speech[L_FRAME];
char Packed[10];
Word16 vad_enable = 1;
Word16 frameErase = 0;

G729ABENC_Handle  encHandle;
IG729ABENC_Fxns   fxns_enc;
G729ABENC_Params  params_enc;
G729ABENC_Status  status_enc;
G729ABENC_DynamicParams  DynamicParams_enc;
XDM1_SingleBufDesc inBufs_enc;
XDM1_SingleBufDesc outBufs_enc;
G729ABENC_InArgs inArgs_enc;
G729ABENC_OutArgs outArgs_enc;

G729ABDEC_Handle  decHandle;
G729ABDEC_Handle  handle_dec;
IG729ABDEC_Fxns   fxns_dec;
G729ABDEC_Params  params_dec;
G729ABDEC_Status  status_dec;
G729ABDEC_DynamicParams  DynamicParams_dec;
XDM1_SingleBufDesc inBufs_dec;
XDM1_SingleBufDesc outBufs_dec;
G729ABDEC_InArgs inArgs_dec;
G729ABDEC_OutArgs outArgs_dec;

inBufs_enc.buf = (XDAS_Int8 *)speech;
inBufs_enc.bufSize = L_FRAME*sizeof(Word16);
outBufs_enc.buf =(XDAS_Int8 *) Packed;
outBufs_enc.bufSize = 10;

DynamicParams_enc.isphenc1_DynamicParams.size =
sizeof(IG729ABENC_DynamicParams);
status_enc.isphenc1_status.size = sizeof(G729ABENC_Status);

fxns_enc = G729ABENC_ADT_IG729ABENC;
params_enc = G729ABENC_PARAMS;

inBufs_dec.buf =(XDAS_Int8 *) Packed ;
inBufs_dec.bufSize = 10;
outBufs_dec.buf = (XDAS_Int8 *) speech;
outBufs_dec.bufSize = L_FRAME*sizeof(Word16);

DynamicParams_dec.isphdec1_DynamicParams.size =
sizeof(IG729ABDEC_DynamicParams);
status_dec.isphdec1_status.size = sizeof(G729ABDEC_Status);

fxns_dec = G729ABDEC_ADT_IG729ABDEC;
```

```
params_dec = G729ABDEC_PARAMS;

encHandle = G729ABENC_create(&fxns_enc, &params_enc)
decHandle = G729ABDEC_create(&fxns_dec, &params_dec)

inArgs_enc.isphenc1_inArgs.size = sizeof(IG729ABENC_InArgs);
outArgs_enc.isphenc1_outArgs.size = sizeof(IG729ABENC_OutArgs);
inArgs_enc.VADEnable = vad_enable;
G729ABENC_encode(encHandle, &inBufs_enc, &outBufs_enc, &inArgs_enc,
&outArgs_enc);

inArgs_dec.isphdec1_inArgs.size = sizeof(IG729ABDEC_InArgs);
outArgs_dec.isphdec1_outArgs.size = sizeof(IG729ABDEC_OutArgs);
inArgs_dec.frameErase = frameErase;
inArgs_dec.isphdec1_inArgs.frameType =
                        outArgs_enc.isphenc1_outArgs.frameType
G729ABDEC_decode(decHandle, &inBufs_dec, &outBufs_dec, &inArgs_dec,
&outArgs_dec);

G729ABENC_delete(encHandle);
G729ABDEC_delete(decHandle);
```

3.5 C++ API

The C++ API is defined in g729abdll.h, and copied here for convenience. The C++ API is a simply a wrapper for the ADT API. The Encode method corresponds to the G729AB_ADT_Encode function in the ADT API. The Decode function corresponds to the G729AB_ADT_Decode function in the ADT API. Please refer to the section of this document that describes the ADT API for details of input and output. Note that since this is a C++ class, the instance data and scratch data are handled within the class. The user need not instantiate this data or make any references to it.

There is one difference between the C++ API and the ADT API. The C++ API does not include a VAD Flag Instead, the number of bytes of compressed data output by the encoder or passed into the decoder indicates what type of frame it is (VAD Flag) as follows:

Number of Bytes == 0: Null Frame (VAD Flag = 0)
Number of Bytes == 2: SID Frame (VAD Flag = 2)
Number of Bytes == 10: Voice Frame (VAD Flag = 1)

```
class MY_APP_API G729AB
{
public:
      G729AB( );
      ~G729AB( );
      void Encode(ADT_Int16 *pIn, ADT_UInt8 *pOut, ADT_UInt8
*pNOutputBytes, ADT_UInt8 VADEnable);
      void Decode(ADT_UInt8 *pIn, ADT_Int16 *pOut, ADT_UInt8
NInputBytes, ADT_UInt8 FrameEraseFlag);
      G729ENC_ADT_Instance_t EncInstance;
      G729DEC_ADT_Instance_t DecInstance;
      G729ENC_ADT_Scratch_t EncScratch;
```

```
      G729DEC_ADT_Scratch_t DecScratch;

};
```

3.5.1 Sample Code

```
#include "g729abdll.h"

G729AB MyG729AB;  //Instantiate and in initialize the codec
#define L_FRAME 80
Word16 speech[L_FRAME];
ADT_UInt8 packed[10];
ADT_UInt8 vadEnable = 1;
ADT_UInt8 vadFlag = 0;
Word16 frameErase = 0;
ADT_UInt8 nBytes;
MyG729AB.Encode(speech, packed, &nBytes, vadEnable);


MyG729AB.Decode(packed, speech, NInputBytes, frameErase);
```